

TaskSense: 一种基于大语言模型的异构传感器系统任务分配类翻译方法

Kaiwei Liu¹, Bufang Yang¹, Lilin Xu¹, Yunqi Guo¹, Guoliang Xing¹, Xian Shuai², Xiaozhe Ren², Xin Jiang², Zhenyu Yan¹ ¹香港中文大学, 香港特别行政区, 中国²华为技术有限公司诺亚方舟实验室, 香港特别行政区, 中国

摘要

越来越多的环境, 如智能家居和工厂, 正在配备多传感器系统, 以实现多样化的智能应用。然而, 大多数现有的传感器协调系统需要手动预定义规则, 限制了其处理灵活复杂任务的能力。虽然最近的方法利用大语言模型 (LLMs) 与外部API交互, 但它们难以完全理解实际传感器系统的能力和依赖关系。本文介绍了TaskSense, 一个响应复杂用户查询、协调多传感器系统的新型系统。TaskSense引入了一种传感器语言, 能自动将传感器系统的能力和依赖关系转换为大语言模型可理解的词汇和语法规则。随后, 它结合大语言模型, 利用这种传感器语言将用户意图解释为传感器系统的可执行任务计划。同时, TaskSense检查用户查询的可解性, 并验证任务计划依赖关系的正确性。为了进一步增强鲁棒性, TaskSense集成了一个动态计划执行机制, 该机制根据传感器数据可用性、数据质量和执行结果的实时反馈来调整计划。TaskSense已部署在真实世界的智能家居系统上, 使用了六种流行的大语言模型。该系统在涉及9种传感器系统、超过60个API、170项任务和5种数据模态的4个场景中进行了评估。结果表明, 与基线方法相比, 在使用相似数量令牌的情况下, TaskSense实现了高达2×的规划准确率提升, 并将答案准确率提高了75%。

CCS概念

• 以人为中心的计算 → 普适与移动计算; • 计算方法学 → 规划与调度; • 计算机系统组织 → 嵌入式与信息物理系统。

关键词

传感器系统, 大语言模型, LLM智能体, 物联网

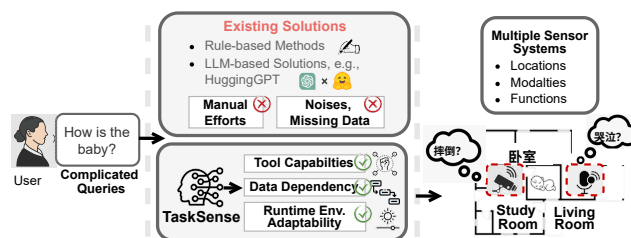


图1: TaskSense系统的一个示例场景。它可以通过协调多传感器系统来处理用户的复杂查询。

ACM参考格式:

Kaiwei Liu, Bufang Yang, Lilin Xu, Yunqi Guo, Guoliang Xing, Xian Shuai, Xiaozhe Ren, XinJiang, Zhenyu Yan. 2025年.TaskSense: 一种用于异构传感器系统任务分配类翻译方法。发表于第23届ACM嵌入式网络传感器系统会议 (SenSys '25), 2025年5月6日至9日, 美国加利福尼亚州尔湾市。ACM, 美国纽约州纽约市, 13页。https://doi.org/10.1145/3715014.3722070

1 引言

传感器系统已广泛应用于人类生活的各个方面, 支持着医疗保健和工业物流等多样化的智能应用和任务 [40, 63]。预计到2030 [2]年, 传感器系统的市场规模将增长至552亿美元。为了支持多样化的智能应用, 多传感器系统正越来越多地被部署并共存于同一环境中。大多数现有的传感器系统都是为特定任务而设计的, 其功能在部署后即保持固定 [50, 61, 63]。最近, 像 Home Assistant [9] 这样的开源生态系统已经成熟, 集成了超过一千种不同的设备和服务。这些生态系统催生了一个愿景: 即有可能控制来自不同制造商的异构智能设备来处理复杂的任务。然而, 这些生态系统仅支持固定的协调规则, 例如 Home Assistant 平台中的自动化脚本, 这些规则需要人工预先手动定义。因此, 由于智能设备存在太多潜在的组合, 这些系统无法扩展以处理灵活且复杂的任务。

近期, 基于大语言模型的研究, 如HuggingGPT [49]和 ControlLLM [35], 已证明大语言模型能够理解用户查询并协调多个AI模型以处理复杂任务。作为中央控制器, 大语言模型有潜力编排异构传感器系统, 克服传统人工智能模型局限于单一、预定义任务的限制。凭借其强大的理解能力, 大语言模型能够理解以自然语言表达的灵活且复杂的查询。



本作品采用知识共享署名 4.0 国际许可协议进行许可。SenSys '25, 2025年5月6日至9日, 美国加利福尼亚州尔湾市© 2025年 版权所有/作者所有。

ACM ISBN 979-8-4007-1479-5/25/05
https://doi.org/10.1145/3715014.3722070

根据用户意图生成正确的计划, 并将计划执行结果转化为用户友好的响应。然而, 现有基于大语言模型的协调方案仅适用于AI模型或算法, 难以协调真实传感器系统。由于缺乏对现实世界传感器系统能力及其功能模块间数据依赖关系的了解, 现有方法无法准确判断用户查询的可解性, 也无法以正确方式协调传感器系统。它们在现实环境的不确定因素下也难以保持稳定的性能。图1展示了一个示例场景: 一个家庭配备了视频监控系统和智能音箱。用户发送查询, 要求监控家中婴儿的潜在异常状态。如果检测到婴儿摔倒时哭泣, 传感器系统应向用户发出警报。此类查询需要传感器系统的多样化组合, 且具有高度动态性。其他示例包括, “昨天房间里有人吗?” 和“我的钥匙在哪里?”。

为填补这一研究空白, 本文探索了一种LLM驱动的系统的的设计, 该系统通过充分理解一个或多个传感器系统的能力以及所涉及工具之间的依赖关系来协调它们。本文中的工具指的是传感器系统内具有特定功能的模块, 例如数据处理和传感器控制。然而, 设计这样的任务分配系统面临若干挑战。首先, 一旦传感器系统部署完毕, 协调系统必须能够理解工具多样化的功能。这些功能可能包括专门的AI模型、数据库操作、传感器控制或数据传输。理解每个工具的能力限制及其复杂的相互依赖关系, 对大语言模型构成了重大挑战。例如, 一个监控摄像头系统可能检测到异常行为, 但可能不具备识别个体的能力。同时, 一个面部识别工具可能只接受裁剪后的人脸图像作为输入, 而不是传感器直接捕获的完整图像。其次, 现实环境的变化会显著影响数据质量和可用性, 可能导致协调失败。协调系统难以在运行时通过动态替换工具来自动生成替代计划以完成同一任务。

在本文中, 我们介绍了TaskSense, 这是一种基于翻译的方法, 使大语言模型能够通过一个称为传感器语言的专用框架与传感器系统交互。如图1所示, 我们首先开发了一种新颖的传感器语言, 以帮助大语言模型理解传感器系统的能力和数据库依赖关系。一旦这些系统部署完毕, 我们的方法会自动将其功能翻译成大语言模型能够理解的词汇和语法。当用户提交查询时, TaskSense首先通过可解性检查评估其可解性, 然后基于传感器语言解释用户意图, 并将其翻译成传感器系统可以执行的可执行计划。接着, TaskSense执行语法检查, 以检测计划中的任何依赖错误。在执行过程中, TaskSense采用动态计划适应机制, 根据传感器数据质量、数据可用性和执行结果的反馈, 在运行时调整计划。这使得系统能够在现实环境中保持高准确率和适应性。最后, TaskSense

将传感器系统的输出翻译回自然语言, 为用户提供清晰的响应。

我们将本工作的贡献总结如下:

- 我们开发了TaskSense, 这是首个类翻译方法, 使大语言模型能够协调异构传感器系统以完成用户查询中的复杂任务。
- 我们提出了一种新颖的传感器语言, 它定义了词汇集和语法, 使大语言模型能够理解传感器系统中功能模块的能力边界与相互关系。这有助于TaskSense自动生成可执行计划来协调传感器系统, 并确保任务可解性与数据依赖性。
- 我们为传感器系统协调设计了一种动态计划适应机制, 以应对运行时的环境不确定性。它允许TaskSense根据传感器故障、数据质量和执行结果, 动态调整计划执行路径。
- 我们在4个数据集上实现了TaskSense, 涵盖9个传感器系统、超过60个API、170项任务和5种数据模态。评估结果表明, 与最先进解决方案相比, 在使用相似数量令牌的情况下, TaskSense的规划准确率最高可达其3倍, 响应准确率最高可达其1.75倍。

2 相关工作

2.1 传感器系统任务分配

各种智能传感器系统已融入我们的日常生活 [11, 60, 66]。传统的传感器系统依赖于基于规则的任务执行策略。ADMarker [39] 在老年人家庭中结合多种传感器和AI模型, 以检测早期阿尔茨海默病。Kratos+ [51] 使用策略协商算法来管理多个传感器, 处理各种用户请求而不会产生冲突。然而, 这些解决方案仅限于预定义和固定的任务, 无法适应动态的用户需求。最近的研究利用大语言模型来解决嵌入式系统中的复杂任务。[14, 48] 中的研究使用大语言模型向AI模块和物联网设备分配任务, 以满足多样化的用户需求。Sasha [25] 采用大语言模型来管理家庭设备, 并根据开放式指令生成行动计划。然而, 这些解决方案在为传感器系统分配任务时未考虑环境不确定性, 这限制了其在现实世界应用中的鲁棒性。

2.2 大语言模型用于任务规划

大语言模型智能体。一些研究提出使用大语言模型作为决策中心来调用外部工具以解决复杂任务。ToolFormer [45]表明大语言模型可以学会使用外部工具的API来生成更可靠的答案。HuggingGPT [49] 和TaskMatrix.AI [33]使用大语言模型作为控制器来提出任务解决方案计划, 然后为每个步骤选择相应的工具。为了从所有可能的计划中找到最佳选择, ToolLLM [44] 和ControlLLM [35] 分别采用基于树和基于图的方法进行最优计划搜索。然而, 这些系统仅专注于调用通用工具, 如计算器、维基百科和专为特定任务设计的AI模型, 并未考虑实际传感器系统中的复杂环境和数据依赖关系。

提升大语言模型规划质量。自检 [36] 使用一个额外的大语言模型作为验证器, 以识别其自身

逐步推理过程中的错误。ToolLLM [44] 和 KwaiAgents [42] 通过在高质量指令数据集上进行微调, 提升了大语言模型的规划能力。此外, 许多研究采用基于反馈的策略来改进计划质量。

TaskMatrix.AI [33] 和 Sasha [25] 利用来自用户的反馈来改进大语言模型的计划质量。其他研究也利用错误消息作为反馈来提升计划质量。Voyager [58] 和 LLMind [14] 使用程序执行错误作为反馈来改进大语言模型的计划质量。一些工作还尝试通过来自客观环境 [68, 70] 的观察来增强大语言模型。然而, 这些方法要么需要额外的验证器进行评估, 要么依赖执行错误和人工反馈来改进计划质量, 未能充分利用传感器系统中的信息。

2.3 用于传感器系统中开放式问答的大语言模型

将传感器数据适配到大语言模型嵌入空间。 M4 [69] 和 OneLLM [21] 使用大量多模态传感器数据来训练适配器。这些适配器可以将原始传感器数据映射到大语言模型的嵌入向量空间中, 从而支持基于传感器数据的开放式问答。然而, 将多样化的传感器数据模式适配到自然语言的嵌入空间需要大量的训练数据, 并且难以实现, 特别是对于像惯性测量单元这样复杂且信息稀疏的模式。

检索与总结。 许多研究探索了大语言模型在嵌入式系统 [41, 62, 64] 内进行传感器数据推理的能力。LLMTrack [67] 使用思维链 [27] 提示来分析惯性测量单元数据以进行轨迹识别, 而 LLMsense [41] 则使用大语言模型对人类活动识别模型生成的日常活动日志进行高层推理。此外, 一些研究利用检索增强生成技术 [23, 62] 来访问外部知识库, 例如患者的日常可穿戴数据 [65]。然而, 这些系统面临两大局限: 一些系统需要持续执行各种特定任务模型来更新数据日志, 这在计算资源和灵活性方面带来了重大挑战; 而另一些系统则主要侧重于传感器数据的理解与推理, 忽略了更广泛的任务类型。

3 研究动机

本节审视当前为传感器系统分配任务的方法, 该方法涉及三个步骤: 基于用户查询生成工具调用计划、执行计划, 以及根据执行结果交付响应。通过对现有方法的测量研究以及我们真实世界测试平台的洞察, 我们形成了设计目标。

3.1 基于查询的规划

为了解现有传感器系统处理现实世界用户查询的能力, 我们使用一个查询集来测试最先进 (SOTA) 方法 HuggingGPT [49] 在家庭养老场景中的应用。在此场景中, 包括家庭监控系统、深度动作识别系统和声音检测系统在内的多传感器系统被部署在老年人的家中, 以监测和记录其日常活动。我们使用 HuggingGPT 来理解源自 MDS-UPDRS [20] 的关于人类活动和情绪的临床查询, 然后生成工具调用计划。

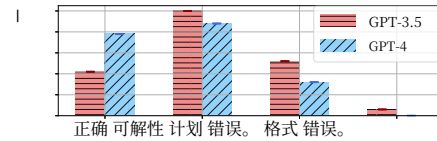


图2: 在老年护理场景中使用HuggingGPT进行任务规划时的错误分布。

我们总共构建了38个老年人护理查询。给定来自3个不同传感器系统的19个工具, 大语言模型需要从这些工具中进行选择, 为每个查询制定一个计划。

图2显示了GPT-4 [5] 和GPT-3.5 [29], 所犯错误的分布情况, 这些错误主要来自以下几个方面:

1. 误判可解性: 当前方法常常无法识别工具集的能力限制, 并识别出超出其能力的用户查询。超出工具集能力的用户查询通常发生在两种情况下。首先, 工具集可能缺少查询所需的特定功能。例如, 对于像“总结鲍勃的血糖情况”这样的查询, 如果工具集缺少血糖监测工具, 系统就无法处理。其次, 即使工具集包含相关工具, 其标签设置也可能未覆盖查询的目标, 例如, 一个包含“坐着”和“跳舞”标签但未包含“下棋”标签的人类活动识别工具。在这些情况下, 系统不应生成误导性的计划, 而应通知用户无法满足该查询。然而, 准确理解每个查询的可解性具有挑战性, 因为工具集的能力边界很复杂。

2. 错误计划: 另一种主要的错误类型涉及生成的计划包含不正确的依赖关系或工具。每个工具都有特定的功能和不同的输入输出要求, 这意味着工具之间的数据依赖关系是受限的。大语言模型可能会通过引入不正确的依赖关系来产生不可行的计划。例如, 一个需要人脸图像作为输入的面部表情识别工具, 在计划中只能跟在人脸检测工具之后。如果它跟在面部情绪识别或说话人日志工具之后, 而这些工具并不输出检测到的人脸, 那么执行结果将是错误的。除了依赖关系问题, 生成的计划还可能包含错误的工具。当大语言模型从工具集中选择了不合适的工具或生成了不存在的工具时, 就会发生这些错误。例如, 针对一个关于睡眠质量的查询, 生成的计划可能错误地包含了一个跌倒检测工具。

图2中最高比例的错误是由于模型未能准确识别超出工具集能力边界的查询。不正确的依赖关系源于模型难以确定工具之间的有效依赖关系。此外, 错误的工具选择通常源于模型在理解多样化的查询表达风格方面面临的挑战。这些限制导致现有方法难以生成高质量的计划。

3.2 Plan Execution and Responding

制定好执行计划后, TaskSense应执行该计划并响应用户查询。为了解计划在现实环境中的执行质量, 我们使用 HuggingGPT [49] 的框架, 在一个智能家居场景中进行了实验。

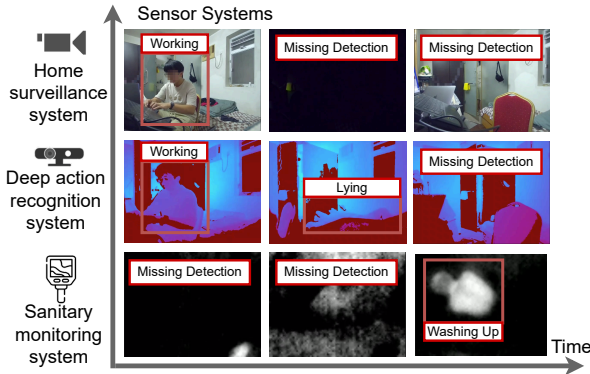


图3：关于环境对传感器系统影响的示意图。虽然LLM规划器会根据用户查询制定计划，但诸如传感器噪声和遮挡等因素会影响执行结果。

我们基于用户查询分析了计划的执行情况，并考虑了可能影响结果的环境不确定性。在本实验中，我们针对同一个用户查询“鲍勃今天是否过度工作且没有足够的洗手间休息？”，在三个不同的传感器系统上分别执行了三个不同的计划。图3展示了计划执行结果中的几个时间段。结果表明，当环境变化时，每个计划的表现各不相同。例如，在基于RGB摄像头的家庭监控系统上执行的计划，由于光照条件差和遮挡，在后两个时间段未能产生正确结果。同样，在深度动作识别系统和卫生监测系统上执行的计划，也分别由于遮挡和目标离开画面，在最后两个和最初两个时间段未能得出准确结果。这表明，尽管计划本身正确，但其执行极易受环境因素影响。此外，不同计划执行路径上的环境因素差异显著。然而，我们注意到，尽管其中一些系统在某些时间段未能检测到目标，但其他系统仍可完成检测，这体现了不同传感器系统之间高度的互补性。我们的研究强调了影响计划执行的三种主要环境因素类型：

- 数据缺失：**计划内的工具执行通常需要传感器数据输入。如果传感器因硬件故障而失效或意外关闭，所需数据可能无法被记录，从而导致执行失败。
- 数据噪声：**诸如影响RGB摄像头的光照条件变化，或影响语音采集的噪声水平等因素，会降低传感器数据的质量。数据噪声会在计划执行结果中引入误差。
- 内容相关问题：**诸如物体移出相机画面、遮挡或个体处于挑战性姿态等问题，仅通过评估数据噪声水平是无法检测的，但它们也会显著影响结果的准确性。

这些观察结果表明，当前方法在处理现实世界场景中的动态环境因素时面临着挑战。为了解决这些问题，利用不同传感器间的互补性

可以成为一个有前景的策略。例如，虽然光照条件可能影响使用基于RGB相机的工具生成的计划的性能，但它们对使用基于深度相机的工具制定的计划影响甚微。此外，通过在家庭中的不同位置和角度部署多个人体检测传感器，可以根据用户的位置在不同传感器生成的计划之间切换，从而确保检测结果。因此，在不同计划执行路径之间自适应地切换，可以减轻环境不确定性的影响，并增强系统的整体鲁棒性。

此外，在计划执行后，现有方法在响应生成阶段也面临挑战。多传感器系统在执行过程中可能产生大量的结果数据。然而，处理大量数据序列并进行复杂计算对大语言模型生成面向用户的响应构成了挑战，[72]，导致遗忘问题或诸如工具间错误数据关联之类的幻觉。为了理解此问题，我们构建了30个测试查询，并使用GPT-4来理解计划执行结果，从而为每个查询生成答案。在测试集中，有5个失败案例是由于不同工具间结果关联错误，另有3个案例呈现了不全面的答案。因此，有必要实施策略以确保生成答案的准确率，以应对与大语言模型相关的局限性。

4 系统概览

本文中，我们介绍了传感器语言，并提出了一种异构传感器系统任务处理方法——TaskSense，它利用大语言模型来协调传感器系统，以在动态环境中理解和响应用户查询。图4概述了TaskSense的设计。TaskSense首先采用一种基于大语言模型的传感器系统自动注册方法（§ 5.1）来初始化词汇集、语法和种子示例，从而减少系统初始化和更新所需的手动工作。当TaskSense收到用户的查询时，它首先基于传感器语言和检索到的示例解释用户的意图，将其翻译成可执行工具调用计划以协调传感器系统（§ 5.2）。在协调计划生成后，TaskSense采用动态计划适配机制（§ 5.3）在运行时进一步调整计划执行路径，确保其能适应环境变化并对嵌入式系统中的干扰具有鲁棒性。最后，TaskSense基于计划执行结果（§ 5.4）将传感器语言翻译回自然语言反馈。

5 DESIGN OF TASKSENSE

本节介绍TaskSense的设计细节。第5.1节介绍了将TaskSense应用于新场景时所需的组件及其初始化过程，包括传感器语言定义和示例库。第5.2节和第5.3节分别阐述了计划检查和动态计划适应技术。第5.4节介绍了响应生成模块。

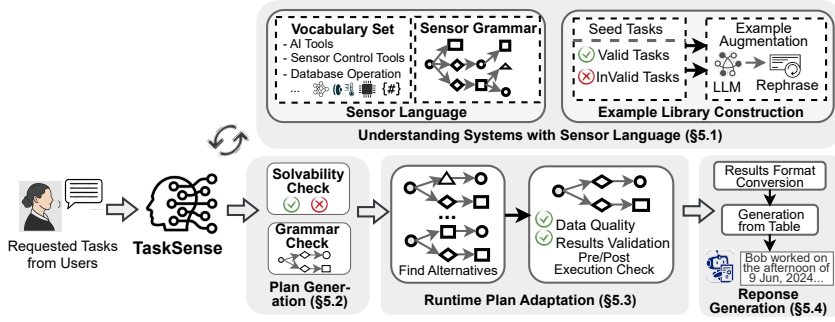


图4: TaskSense的系统概览。

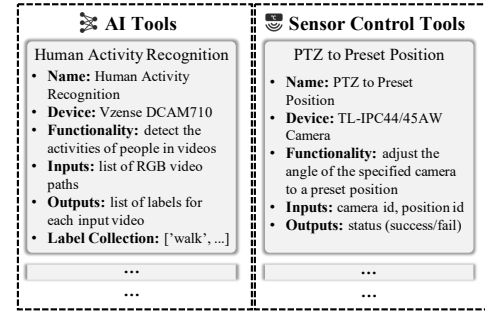


图5: 词汇集示例。

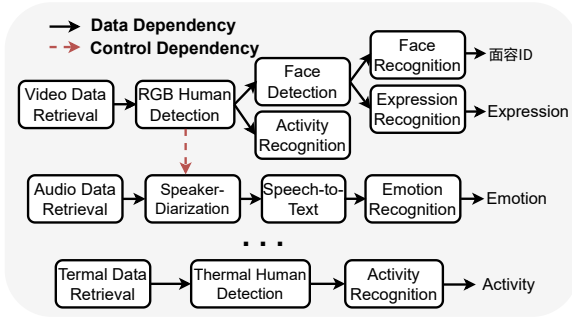


图6: 传感器语法示例。

5.1 理解系统与传感器语言

5.1.1 传感器语言定义。在TaskSense中, 用户查询以自然语言形式输入, 无法直接用于执行传感器系统中的工具。为了弥合这一差距, 我们开发了一种类似翻译的解决方案, 利用大语言模型将用户意图转换为可执行工具调用计划, 这运用了大语言模型在语言翻译任务中的卓越能力 [15, 28]。我们将工具及其依赖关系概念化为一种语言, 称为传感器语言。在此范式中, 工具被建模为传感器语言中的词汇, 依赖关系被建模为语法, 而计划则像是遵循语法规则组合词汇形成的句子。传感器语言包含两个关键定义: 词汇集和语法规则。它们与检索到的示例 (第5.1.2节) 一同包含在LLM提示中, 使大语言模型能够有效地翻译用户意图。

传感器词汇集。它定义了所有工具以及大语言模型在创建可执行计划时可用的基本元素的详细信息。如图5所示, 对于每个工具, 每个词汇项包含以下内容: 类别 (例如, 小型AI模型、传感器控制)、工具名称、模态 (例如, RGB、IMU)、设备 (例如, Vzense DCAM710)、功能描述、输入 (例如, 热成像图像)、输出 (例如, 标签、视频) 以及输出标签设置 (例如, [“坐着”、“跑步”、...])。

传感器语法。它定义了每个计划应遵循的工具间的依赖规则。该传感器语法由一组有向无环图组成, 其中节点代表工具, 边代表工具间的有效依赖关系。当作为大语言模型提示的一部分时, 这些图将被转换为

文本格式, 包含节点列表 (例如, [“人脸识别”、“人脸检测”、...]) 和边列表 (例如, [“人体检测 -> 跌倒检测”、...])。每个有效的计划都是传感器语法的一个子图。如图6所示, 传感器语法中存在两种类型的依赖关系: 数据依赖关系和控制依赖。

数据依赖关系定义了工具之间的数据输入输出关系, 某些工具需要其他工具的输出作为输入, 从而约束了可执行计划。这些依赖关系用于验证大语言模型生成的计划 (更多细节见第5.2节)。另一方面, 控制依赖与数据依赖不同, 主要定义工具潜在的执行前提条件。两个工具之间的控制依赖规定了一个工具可以执行的条件, 这取决于另一个工具的输出。例如, 在图6中, 只有当RGB人体检测工具检测到至少一个人时, 说话人日志化工具才会被触发。一旦生成的计划通过数据依赖性验证, 控制依赖会在计划执行前自动插入到计划中。

传感器词汇表和语法对于大语言模型将用户查询转化为可执行计划至关重要。词汇表定义了工具集的能力边界, 而传感器语法规则规定了工具间的依赖规则。它们将分别用于可解性检查 (§5.2.1) 和语法检查 (§5.2.2) 模块, 以提高规划质量。当将TaskSense应用于新的应用场景时, 需要首先构建词汇集和语法。现有解决方案, 如HuggingGPT [49] 和Sasha [46], 需要人工努力来创建工具集。相比之下, TaskSense实现了自动初始化。具体来说, TaskSense首先构建传感器词汇集。用户只需提供软件工具 (例如, 人体检测和活动识别) 的源代码和/或由制造商提供的传感器系统API的详细描述 (例如, 视频捕获和人员存在)。TaskSense在提示中指定了新词汇的格式: 一个JSON对象列表, 每个JSON对象代表一个词汇。每个JSON对象具有以下键: “category”、“tool name”、“modality”、“description”、“input”、“output”和“label setting”。此外, TaskSense还提供了5-6个JSON形式的预定义词汇示例, 以帮助大语言模型理解格式。然后, TaskSense将包含新工具的源代码/详细描述以及示例的提示发送给大语言模型, 以生成所有新工具的词汇。在构建新词汇之后, TaskSense开始构建新语法规则。同样, 它在

提示中指定, 语法规则应组织为一个字符串列表。每个字符串包含由箭头连接的两个工具的名称, 例如“FaceDetection -> Face Recognition”, 表示一种依赖关系。TaskSense还将遵循此语法规则格式的字符串列表作为示例, 以及新工具的词汇表, 一并包含在提示中。借助此提示, 大语言模型为新工具生成语法规则。之后, TaskSense将根据每对工具的输入和输出检查每个新生成的语法规则是否有效。用户也可以手动添加或删除新词汇或语法规则。

5.1.2 示例库。 仅提供词汇集和语法规则不足以学习传感器语言。在机器翻译中, 示例库(即平行语料库)通常用于组织翻译示例以训练模型 [24, 26, 30]。如表格1所示, 我们将此方法适配为一种基于大语言模型的传感器语言方法, 将每个示例设计为一个查询-计划对。每个查询-计划对包含一个用户查询示例以及解决该查询的相应计划。大语言模型从这些示例中学习如何将用户查询分解为工具调用步骤, 并将其组织为可执行计划。示例库有两个关键作用: (i) 帮助大语言模型准确理解工具集的能力边界, 以识别不可解查询; (ii) 提高规划对用户不同表达风格的鲁棒性。为实现第一个目标, TaskSense在库中同时包含可解示例和不可解示例。为实现第二个目标, TaskSense对种子示例进行分类和采样, 然后利用大语言模型自动扩展它们。

种子示例分类。 我们将最初构建的示例称为种子示例。种子示例的主要目的是确保示例库具有广泛且多样的覆盖范围。鉴于这种多样性对于提升指令调优性能至关重要 [12, 71], 我们建议通过分类采样来扩展种子示例。可能的查询类别根据具体应用进行组织, 并为每个类别创建示例。例如, 针对家庭养老场景 [39], 我们组织了“活动”、“认知”、“情绪”等类别。在每个类别内部, 种子示例的组织力求尽可能差异化。分类和类内多样性共同提升了系统在面对未见用户查询时的性能。

可解与不可解示例。 在现实场景中, 我们的系统可能会收到超出工具集能力边界的查询。系统需要识别这些不可解查询, 从而避免产生误导性响应。然而, 仅按类别创建示例不足以让系统准确评估查询的可解性。为了解决这个问题, 我们改进了示例库的设计, 为每个类别同时创建可解和不可解示例。一个可解示例包含一个可处理的查询及其对应的计划, 而一个不可解示例则包含一个无法处理的查询和一个空计划, 表明没有可用的工具调用计划。这种方法显著增强了大语言模型识别查询可解性的能力。

示例库增强。 用户通常表现出多样的表情风格, 这可能影响大语言模型输出的准确率。为了解决这个问题, 我们提出利用大语言模型来多样化示例中查询的表情风格。对于每个种子示例, 我们将查询输入给一个大语言模型, 并指示其生成多个语义上

Table 1: Example library in TaskSense.

Type	Query	Planning Result
Activity	Did Bob show repeated action yesterday?	{ "query-video-rgb" ..., "human-activity-recognition-rgb", "activities" }
Emotion	How was Bob's mood today?	{ "query-video-rgb" ..., "expression-recognition-rgb", "expressions" }
Language	Did Bob mention his travel yesterday?	{ "query-audio-data" ..., "speaker-recognition", "speaker ID" }
不可解-able	鲍勃本周是否发烧或感到疲劳?	{ "不可解" }
...	...	...

等效但风格不同的查询。每个新查询都与种子示例的计划配对, 从而为示例库创建一个新示例。基于大语言模型指令重写的优势 [32, 59], 此方法能有效增强示例库。

与构建新词汇和语法类似, TaskSense 也自动化了种子示例的构建。它规定示例的格式应为一个 JSON 对象 对的列表。在每对 JSON 对象 中, 一个包含值为“用户”的“role”键和值为用户查询的“content”键。另一个 JSON 对象 的“role”键为“助手”, “content”键则为针对该用户查询的相应计划。这种格式演示已包含在提示中。与构建词汇和语法规则类似, TaskSense 在提示中也包含了少量遵循此格式的预定义示例。先前步骤中生成的新工具的词汇和语法规则也同样被包含在内。然后, 该提示将被发送给大语言模型, 以生成针对新工具的种子示例。此过程为每个种子示例类别单独执行。在生成所有新的种子示例后, TaskSense 会指示大语言模型丢弃无效或过于相似的示例, 然后对其余示例进行语法检查。接着, 它计算剩余示例之间的成对语义相似性, 并根据相似性提供排名, 作为用户通过指定其 ID 手动移除冗余示例的参考。

5.2 Plan Generation

在接收到用户查询后, TaskSense 会使用文本编码器为其生成嵌入向量, 并基于余弦相似度从示例库中检索最相似的示例。TaskSense 会预先计算并存储示例查询的嵌入向量, 以减少系统每次检索的延迟。为确保检索示例的多样性, TaskSense 会均匀地从每个类别中检索示例。这种基于类别的检索策略增强了示例间的多样性, 通过避免示例间过度相似来提高系统的泛化能力。随后, 这些示例会与传感器词汇表、语法规则以及用户查询相结合, 供大语言模型进行计划生成。然而, 大语言模型生成的计划可能包含错误, 例如误判用户查询的可解性以及错误的依赖关系。TaskSense 采用两个关键模块来解决这些问题: 可解性检查和语法检查。

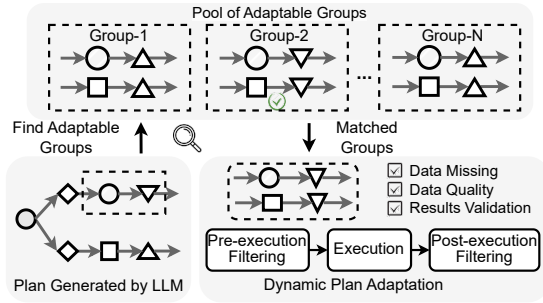


图7：动态计划适配的工作流程。

5.2.1 可解性检查。 有些查询是不可解的，因为其目标标签（例如活动、情绪或事件）超出了工具集的能力边界。为了处理这些不可解查询，除了在示例库中包含不可解示例外，我们还开发了一个基于大语言模型的检查器，根据工具集的能力限制来评估查询的可解性。在评估可解性之前，检查器首先会识别查询是否为开放式查询，即那些不将响应限制在特定标签内、允许任何合理响应的模糊查询（例如，“你对爱丽丝11月14日的锻炼计划感觉如何？”）。开放式查询将跳过进一步的可解性分析。对于具有特定目标标签的查询（例如，“鲍勃在2023年11月13日何时躺在床上？”），检查器会根据工具集的能力评估其可解性。它会检查工具集是否拥有实现目标功能所需的工具，如果具备，则进一步检查这些工具的标签列表是否包含目标标签。针对超出工具集能力范围的标签的查询将被标记为不可解，且不会生成计划，而其余查询则通过检查。

5.2.2 语法检查。 为了识别生成计划中的依赖关系错误，TaskSense使用子图匹配来验证语法正确性。TaskSense首先将语法规则转换为一个包含所有有效工具和依赖关系的有向无环图。该计划也表示为有向无环图，其中每个节点对应一个不同的工具。然后，TaskSense检查该计划的有向无环图是否是代表语法规则的有向无环图的子图。此方法不仅能识别不正确的依赖关系，还能标记任何错误生成的、不在预定义工具集中的工具。

5.3 运行时计划适配

现实环境中的各种动态因素，例如光照和遮挡，可能会降低工具性能。如第3.1节所示，静态计划执行容易受到此类环境因素的影响。然而，词汇集通常包含功能相似的工具，例如面部表情识别和语音情感分类。由于这种冗余性，对于同一个用户查询，可能存在多个可行的计划，这些计划利用部署在不同位置的不同传感器模式或设备。受此启发，我们提出了一种计划适配方法，使TaskSense能够识别计划中的可替换部分，并根据传感器数据的运行时质量以及每条路径的输出，在功能相似的替代方案中动态选择最优执行路径。图7展示了动态计划适配的工作流程，具体描述如下。

5.3.1 可替换部分识别与替代组匹配。 首先，TaskSense利用大语言模型按功能对工具箱中的工具进行分类，将相似的工具分组。然后，TaskSense通过添加每个工具所依赖的最少必要工具，将每个组中的每个工具替换为一个以该工具结尾的计划。这种没有冗余工具的计划被称为替代路径。如果多条替代路径以同一个工具结尾，它们将作为不同路径包含在同一组中。所有这些组将被收集在一起，我们称每个组为一个可适配组，并称该组集合为可适配组池。在运行时，当生成一个计划后，TaskSense会遍历可适配组池，寻找包含的路径是所生成计划的子图的组。匹配到的组将被选为所生成计划中对应路径的可适配组，因为该组中的路径在功能上与其等价。如果匹配到多个组，TaskSense会选择路径最长的那一个。接着，TaskSense从原始计划中移除匹配到的部分，并重复此过程，直到找不到更多的可适配组。这个过程将大语言模型生成的计划转换为一组可适配组和不可替换部分。

5.3.2 执行前过滤。 在执行过程中，各种现实因素会影响传感器数据的可用性和质量。为了在运行时选择合适的路径，我们在执行每个组之前应用执行前过滤，同时考虑数据可用性和质量。具体来说，对于每个具有替代路径的组，TaskSense首先将传感器数据的时间范围划分为相等的间隔，例如一小时。然后，对于每个间隔，TaskSense检查每个替代路径，以检查其所需的传感器数据是否缺失，并评估数据质量。如果数据缺失或信噪比显著过低，则该替代路径在该间隔内将被跳过。

5.3.3 执行后选择。 某些引发不确定性的因素无法通过检查缺失数据或评估数据质量来识别，但它们会影响执行结果。例如，遮挡和有限视野可能导致在某一位置的目标检测失败，但其他类似工具可能有效。为解决此类因素，如果当前路径产生无效的执行结果（例如，空结果），TaskSense会切换到其他替代路径。此评估作为反馈，用于动态调整计划。

此外，为减少计划执行的延迟，TaskSense采用了缓存机制。一个缓存数据库存储了每个工具在特定数据轨迹上的执行结果。当执行计划中的每个工具时，系统会检查缓存数据库，查看输出是否已生成。

5.4 响应生成

计划执行结果随后被发送给大语言模型以生成响应。如第3.2节所述，大语言模型在理解这些结果时面临两个主要挑战：

1. 结果超出令牌限制。
2. 生成响应可能涉及逻辑理解和计算，而大语言模型对此处理不佳。

为应对这些挑战，我们提出一种格式化方法，将执行结果转换为更易读的格式。TaskSense使用唯一标识符（ID）来表示每个工具的结果项，并将原始结果转换为大语言模型易于理解的格式。

表2：数据集详情。

数据集	传感器模式	工具数量	# Seed Examples	# Queries
In-lab HAR	RGB, Depth	6	8	30
DAHLIA	RGB, Depth	6	8	30
Synthetic	RGB, Depth, Audio, IMU	18	10	50
Real-world	RGB, Depth, Audio, 光、红外、Zigbee、湿度、温度	58	10	60

结果导向ID。为了追踪来自不同工具的结果项之间的对应关系，TaskSense为每个工具的结果项分配一个长且随机生成的字符串作为唯一ID。此ID连同工具的原始结果，会作为其前置ID传递给计划中的下一个工具。因此，每个工具都会记录其输入和输出参数，以及前置ID和ID。通过使用这些ID和前置ID，我们可以构建结果树。

结果格式转换。在获得带有结果ID的执行结果后，我们应用最低公共祖先匹配算法 [6] 来识别对象及其对应的标签（如活动、性别和身份），从而构建结果表格。例如，根据图6所示的传感器语法有向无环图，执行结果可能包含人脸和活动识别。通过对对象检测识别最低公共祖先，我们可以确定人员的活动和情绪。最终格式化结果以表格形式组织。

基于格式化结果的响应。由于表格可能超出大语言模型的令牌限制，且大语言模型的数值计算能力有限， [72]，TaskSense 会从结果表格中提取所需信息，类似于SQL查询。

6 实现与数据集

6.1 系统实现

TaskSense的整体架构由服务器上的一个大语言模型以及部署在服务器或边缘设备上的多个工具组成。传感器数据由边缘设备采集。我们利用微软Azure [37], Amazon Bedrock [8] 和 Poe [43] 平台的API来访问各种大语言模型。对于示例的检索，我们利用Cohere [7] 平台的API获取文本嵌入向量，并使用 Annoy [52] 包基于相似性选择示例。

6.2 数据集

我们在表2所示的四个数据集上评估TaskSense：**实验室HAR数据集¹**。该人体活动识别数据集在我们的实验室中采集，包含30名参与者执行30种不同行为的数据。其中三名参与者用于评估，其余用于模型训练。评估中使用了一个 Vzense DCAM710相机系统（RGB和深度）以及6个工具。

DAHLIA数据集 [56]。这是一个常用于人类活动识别的开源数据集。我们使用其中三位个体的数据进行评估，其余数据用于模型训练。该数据集通过三台

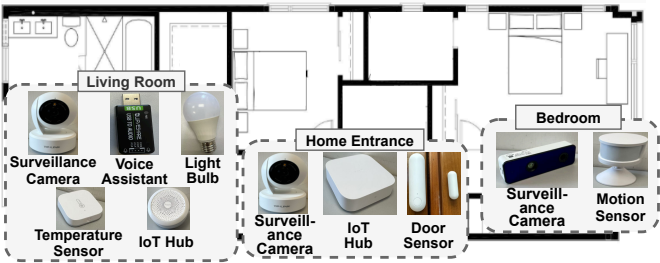


图 8：真实世界数据集的硬件设置。

Kinect v2传感器（RGB与深度）采集。在评估中，我们结合该数据集使用了6种不同的工具。

合成数据集。为了消除工具准确率对TaskSense性能的影响，我们使用大语言模型合成一个虚拟数据集用于模拟测试。该数据集包含两个个体五天的行为，通过提示大语言模型为每个个体在每个时间间隔生成工具输出，合理模拟他们的日常活动。这些模拟输出被直接存储在执行缓存中。传感器系统根据虚拟环境设置模拟了故障条件。我们模拟在此虚拟环境中放置4个不同的传感器系统，并在评估中使用18个工具。

真实世界数据集¹。我们在三个房间内收集了此数据集，使用了8个异构传感器系统，包含12个个体的数据。其中三个个体的数据用于评估。如图8所示，房间1（客厅）包含一个TP-link TL-IPC44AW相机系统（RGB）、一个Waveshare USB转音频模块（音频）、一个小米智能灯和一个Aqara温度传感器。房间2（家庭入口）有一个TP-link TL-IPC45AW安防摄像头（RGB）和一个Aqara门传感器（Zigbee）。房间3（卧室）包含一个小米人体传感器2s（红外）以及一个使用Vzense DCAM710相机（RGB和深度）的行为分析系统，模拟在卧室中为有特殊需求的群体（如老年人）提供监测。本次评估总共使用了58个工具。

由于原始数据集缺乏相应的查询，我们利用来自现实世界调查和医学问卷的查询，包括MDS-UPDRS [20]、PSQI [10]、IPAQ [13]、NBI [18]、ZBI-C [53]、FAS [47]和IADL [31]。此外，我们还创建了额外的查询以确保全面测试。通过这种方式，我们为每个数据集构建了特定的用户查询集和工具集。对于实验室人体活动识别和Dahlia数据集，我们通过随机调整视频亮度并添加高斯噪声来模拟环境因素。**工具实现。**我们整合了来自多个平台的代码和文档，作为数据集中自动系统组件构建的输入。它们包括TP-link IPC控制API [55]、Home Assistant [9]、Vzense DCAM SDK [57]、HuggingFace [17]和FFmpeg [16]，以及来自GitHub的公共模型和我们自己训练的模型。为了使执行模块能够以统一的形式调用工具，我们将每个工具封装为具有标准化输入和输出格式的函数。评估中的工具主要包括三种类型：1. 特定AI模型（例如，跌倒检测）。2. 数据库操作工具（例如，检索传感器数据）。3. 传感器控制工具（例如，设置灯光亮度）。

¹数据收集和已获得作者所在机构审查委员会的批准。

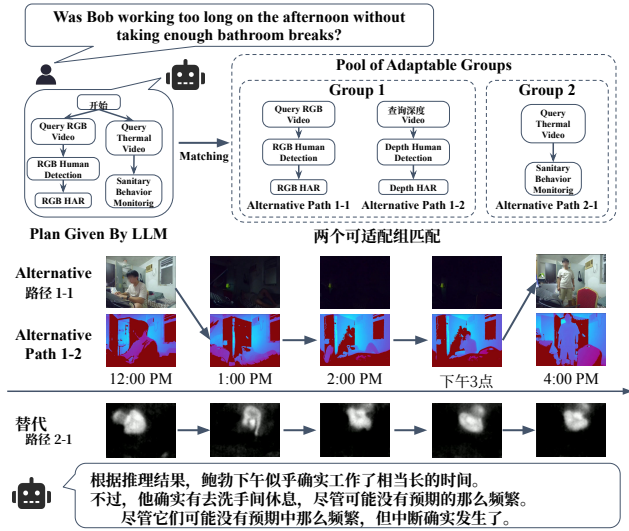


图9: 一个端到端应用示例。

7 评估

7.1 评估指标与基线

7.1.1 评估指标。我们从三个维度评估TaskSense的整体性能。

规划准确率。该指标衡量大语言模型生成正确计划的比例。每个查询的真实计划是手动提供的, 只有当生成计划的工具选择及其依赖关系与真实计划完全匹配时, 该生成计划才被视为正确。如果一个查询存在多个有效计划, 则任何匹配的计划都被认为是正确的。此外, 为了能在计划之间进行更细粒度的比较, 我们还定义了另一个指标**规划得分**, 用于衡量生成计划与真实计划之间的相似度: $S_{score} = \frac{1}{n} \sum_{i=1}^n \frac{2 PR_i RC_i}{PR_i + RC_i}$, 其中 g_i 和 p_i 分别代表真实计划和生成计划中结束节点 (没有后续工具调用的工具调用) 的集合, 且 $RC_i = \frac{|g_i \cap p_i|}{|g_i|}$, $PR_i = \frac{|g_i \cap p_i|}{|p_i|}$, $g_i = \{t_{i1}^g, t_{i2}^g, \dots, t_{ik}^g\}$, $p_i = \{t_{i1}^p, t_{i2}^p, \dots, t_{im}^p\}$ 。通过使用规划得分, 我们的目标是同时衡量每个生成计划相对于对应真实计划的工具覆盖度, 以及生成计划本身内部正确工具的比例。

执行准确率。我们衡量每个查询的正确输出百分比, 然后计算所有查询的平均百分比以确定执行准确率。

响应准确率。它衡量收到正确答案的用户查询的百分比。

7.1.2 基线。我们采用两种具有相似问题设置的方法, 即 HuggingGPT [49] 和 Sasha [46], 作为基线。HuggingGPT 使用大语言模型将复杂任务分解为子任务。它执行简单的格式检查, 不考虑用户查询的可解性或计划的语法正确性, 并且不提供反馈以改进规划质量。与 HuggingGPT 不同, Sasha 涉及额外的步骤来提示大语言模型评估每个用户查询的可解性, 并提高所生成

表3: 不同方法的整体性能 (使用GPT-4作为基础大语言模型)。

数据集	方法	规划 Accuracy	Execution Accuracy	Response Accuracy
In-lab	HuggingGPT	0.80	0.55	0.47
	Sasha	0.86	0.56	0.50
	Ours	0.97	0.75	0.73
DAHLIA	HuggingGPT	0.86	0.43	0.50
	Sasha	0.76	0.42	0.40
	Ours	0.93	0.60	0.70
Synthetic	HuggingGPT	0.66	0.59	0.50
	Sasha	0.32	0.67	0.48
	Ours	0.96	0.72	0.74
真实世界	HuggingGPT	0.55	0.35	0.55
	Sasha	0.66	0.37	0.62
	Ours	0.82	0.64	0.75

计划的质量。为了解决基线在响应生成中的识别问题, 我们对它们应用了相同的结果格式化方法, 使其能够有效响应。此外, 为了公平比较, 我们为 Sasha 提供了与 TaskSense 动态计划适配模块所使用的相同基础信息, 包括数据缺失、数据质量和执行结果, 以便进行基于大语言模型的计划重新生成。

大语言模型。我们使用六种流行的大语言模型作为基线和 TaskSense 的基础模型: 四种商业模型 (GPT-4 [5], GPT-4o [3], Claude-3 [22] 和 Claude-3.5 [22]), 以及两种开源模型 (Llama-3-70B [54] 和 Mistral [1])。商业模型通过官方 API 访问, 而开源模型则通过 AWS Bedrock 访问。我们将大语言模型的温度设置为零, 以最小化输出随机性, 其他设置除非另有说明, 均保留为默认值。

7.2 端到端应用

为演示 TaskSense 的工作流程, 我们在一个智能家居环境中搭建了一个原型。我们将系统部署到一台配备 32 GB GPU 内存的 NVIDIA Jetson AGX Orin [38] 上, 并连接互联网以与大语言模型提供商交互。我们通过此设置进行了概念验证测试, 允许用户查询系统, 以获取由 TaskSense 生成的计划所提供的日常行为洞察。

图9展示了基于参考国际劳工组织 [4] 报告所提出的查询而进行的计划生成、执行和响应步骤。从中午12点至下午4点, 系统根据环境因素动态切换不同的执行路径以进行活动识别。例如, 当下午1点光照水平低于 RGB 相机的阈值时, TaskSense 会切换到使用深度相机的替代执行路径, 从而确保准确的行为检测和正确响应。

7.3 整体性能

TaskSense 的整体性能通过将其规划、执行和响应准确率与四个数据集上的基线模型进行比较来评估。此外, 通过在实验室内部 HAR 数据集上的六种不同 LLM 设置中与基线进行比较, 测试了 TaskSense 的鲁棒性。

表3展示了使用 GPT-4 在不同数据集上的评估结果, 其中 TaskSense 使用超参数 $k = 10$ 作为选定的种子示例数量。结果表明, TaskSense

表格 4: 使用不同大语言模型在实验室数据集上的整体性能。

LLM	方法	规划 准确率	Execution Accuracy	Response Accuracy
GPT-4o	HuggingGPT	0.83	0.51	0.60
	Sasha	0.83	0.47	0.63
	Ours	0.97	0.72	0.70
Claude-3-Opus	HuggingGPT	0.87	0.55	0.57
	Sasha	0.60	0.23	0.53
	Ours	0.97	0.65	0.77
Claude-3.5-Sonnet	HuggingGPT	0.73	0.54	0.50
	Sasha	0.83	0.48	0.73
	Ours	0.93	0.67	0.70
Llama 3 70B (Open-source)	HuggingGPT	0.47	0.27	0.37
	Sasha	0.63	0.48	0.37
	Ours	0.77	0.54	0.57
Mistral Large (Open-source)	HuggingGPT	0.73	0.58	0.47
	Sasha	0.40	0.18	0.30
	Ours	0.97	0.74	0.70

的规划准确率比基线方法高出最多0.64。TaskSense的大部分错误源于将可解查询误判为不可解，这可以通过更精确的查询来缓解。HuggingGPT的规划准确率远低于TaskSense，因为它未对可解性和计划有效性进行任何检查。此外，其示例库中的示例是随机给出的，缺乏足够的多样性。在其错误的测试样本中，误判可解性和生成错误计划两种情况都占很大比例。Sasha无法像TaskSense那样实现可靠的规划性能，尤其是在合成数据集上。这是因为Sasha依赖大语言模型进行计划调整，但大语言模型的理解能力有限，且其输出存在一定随机性。因此，大语言模型很难根据外部环境的复杂反馈信息对生成的计划进行有效调整，从而导致调整错误或不准确。相比之下，TaskSense的动态计划采纳机制使用预定义模块，能更准确、更及时地根据外部信息调整计划。凭借其动态计划采纳机制，TaskSense的表现优于基线方法，将执行准确率提高了0.29，响应准确率提高了0.30。TaskSense的大部分执行和响应错误源于工具不准确性。此外，工具不准确性导致HuggingGPT和Sasha在合成数据集上的计划准确率与执行准确率不一致，其中Sasha生成的正确计划较少，但性能更优的工具较多。TaskSense避免了这种不一致性，因为它总是为每个计划选择最优执行路径。

表格 4 展示了 TaskSense 使用五种不同大语言模型的结果。TaskSense 始终优于基线，这表明其不同大语言模型间具有鲁棒性和有效性。HuggingGPT 倾向于将不可解查询误判为可解（平均 50%），而 TaskSense 对此处理得更好，错误率仅为 4%。Sasha 在处理可解查询时表现不佳，显示出高达 35% 的错误率，而 TaskSense 的错误率较低（10%）。与基线不同，TaskSense 在不同大语言模型间保持了一致的性能。使用 GPT-4o 时，TaskSense 表现最佳，平均规划准确率达到 92.2%。使用 Claude-35-Sonnet 时，它能很好地处理可解查询，但在处理不可解查询时遇到困难。由于 Llama3 的能力有限，

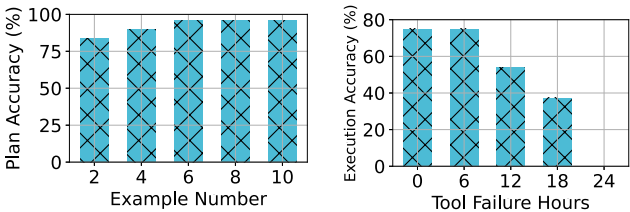


Figure 10: Planning accuracy under different numbers of retrieved seed examples.

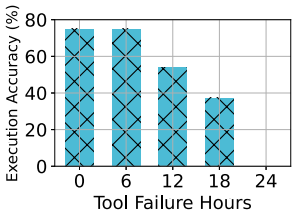


图11: 不同工具故障时长水平下的执行准确率。

TaskSense 与其配合时表现不佳，经常混淆复杂查询。开源模型 Mistral Large 提供了与商业模型相当的结果。

7.4 微基准测试

我们考察了4个微基准测试：(1) 计划生成过程中检索到的种子示例数量的影响，(2) 环境因素干扰水平的影响，(3) 与RAG方法的比较，以及 (4) 种子示例库大小的影响。

7.4.1 检索到的种子示例数量的影响。图10展示了在实验室数据集上，TaskSense的规划性能随检索到的种子示例数量 k 变化的情况。结果表明，其规划准确率随着 k 的增加而提高，但当 k 达到6时趋于收敛，此后检索更多示例带来的性能提升微乎其微。

7.4.2 环境干扰水平的影响。为了评估环境因素干扰水平的影响，我们使用了合成数据集中因环境或设备问题导致工具故障的数据。图11展示了TaskSense的执行性能随一天中工具故障时长的变化情况。结果表明，当故障率保持在合理时长内时，具备动态计划适配设计的TaskSense能有效应对这些变化和故障。例如，TaskSense可以在夜间发生8小时RGB故障时，使用深度相机作为替代方案以维持性能。

7.4.3 与基于RAG的方法比较。TaskSense 通过与两种基于RAG的设置进行比较来进一步评估。第一种设置，规划示例RAG，将RAG应用于示例，它是在TaskSense的基础上开发的，移除了语法和查询可解性检查，并仅在规划期间保留示例检索。第二种设置，工具输出RAG，将RAG应用于推理结果，其设置是持续运行所有工具并将其输出存储在结果数据库中。在此设置下，系统根据用户查询直接从结果数据库中检索相应的工具执行输出，而TaskSense允许用户选择哪些工具持续运行，哪些仅在需要时调用。

我们在实验室HAR数据集上，将我们的方法与规划示例RAG在准确率（图12）方面进行比较，并与工具输出RAG在系统负载（图13）方面进行比较。结果表明，在不同数量的检索示例下，TaskSense的规划得分比规划示例RAG高出多达19%。此外，与工具输出RAG的固定计算负载（图13中的蓝色和

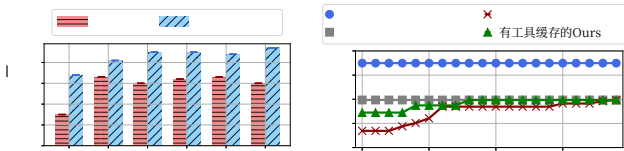


图12: 比较图13: 比较规划示例RAG与工具输出RAG和TaskSense在规划得分方面的对比。TaskSense在累计GFLOPs方面的对比。

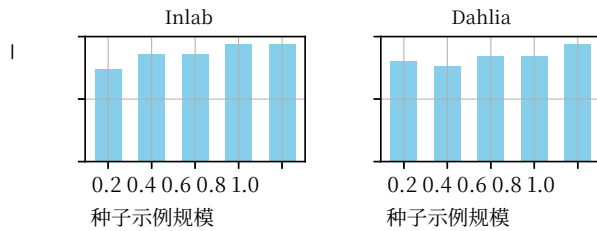


图14: 种子示例规模对规划的影响。

灰色曲线)不同, TaskSense的累计负载随着查询跨越更多时间范围而逐渐增加。采用动态计划适配的工具输出RAG的负载(灰色曲线)可作为TaskSense的上限。此外, 红色与绿色曲线的对比表明, TaskSense中持续运行的工具越多, 会引入更固定的系统开销, 但能更多地减少响应延迟。

7.4.4 种子示例库大小的影响。为了评估种子示例库大小的影响, 我们在实验室内部和Dahlia数据集上通过改变种子示例库规模进行了实验。如图14所示, 我们将§ 7.3中使用的默认种子示例规模设置为‘1.0’, ‘0.8’表示使用80%的种子示例。结果表明, 随着种子示例规模的减小, TaskSense的规划性能下降, 这表明种子示例对于优化TaskSense的规划能力至关重要。

7.5 消融研究

计划生成。表格5展示了TaskSense及其各模块消融变体在合成数据集上使用GPT-4作为基础大语言模型的规划性能。结果表明, 所有模块都对任务规划有所贡献。在这些模块中, 示例库对规划准确率和规划得分的贡献最大(分别为0.28和0.23)。此外, 当移除负例时, 我们的评估显示规划准确率和规划得分分别降至0.92和0.95。这一下降主要是由于错误的工具选择所致, 表明原始大语言模型对工具能力边界的理解有限。

大语言模型的创造力也会影响计划的质量。上述实验是在温度设置为0的情况下进行的。当GPT-4的创造力增强时(温度 = 1.5), 准确率和得分分别下降至0.74和0.80。在这些情况下, 规划验证作为一种补救措施变得有益, 能将准确率提升至0.80, 得分提升至0.87。

表格5: 各组件对规划性能的影响。

方法	规划准确率	规划得分
不含传感器语言指令	0.88	0.91
不含示例库	0.58	0.76
不含检查	0.88	0.91
TaskSense	0.96	0.99

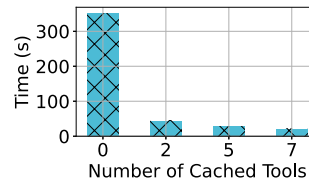


图15: 缓存模块对整体执行延迟的影响。

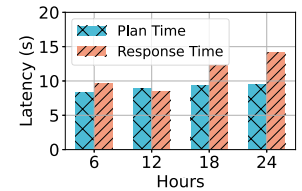


图16: 传感器数据时间范围对规划阶段和响应阶段延迟的影响。

动态计划适配。计划适配设计的有效性在实验室数据集上进行了评估。采用动态规划策略的TaskSense实现了最高的执行准确率, 达到0.82, 而使用固定规划策略配合RGB和深度模态的方法分别实现了较低的准确率0.56和0.60。这表明, 与使用固定规划路径相比, 动态计划适配在提升执行准确率方面是有效的。**结果格式化。**结果格式化对响应准确率的影响在实验室数据集上进行了评估。使用格式化后, 响应准确率提升至73.3%, 而未使用时为66.7%。这表明结果格式化能有效提升系统响应的准确率。

7.6 系统开销

在本节中, 我们评估TaskSense的系统开销, 包括执行延迟和大语言模型的API调用成本。

执行延迟。我们使用实验室数据集评估缓存模块对整体延迟的影响。图15显示, 缓存模块显著降低了系统的执行延迟。在不同数量的缓存工具下, 总时间从360秒降至31秒, 这主要得益于计划执行时间从342秒下降到11秒。同时, 计划生成和响应生成延迟保持稳定, 分别在8-10秒和10-12秒之间波动。此外, 图16展示了不同时间尺度(6、12、18和24小时)下的规划和响应时间。结果表明, 随着数据量的增加, 规划和响应时间也相应上升。

大语言模型API调用成本。我们在实验室数据集上评估了TaskSense的大语言模型API调用成本。表6详细展示了TaskSense各组成部分的令牌成本细分。在我们的方法中, 用于规划和响应阶段的目标描述的固定令牌总数为745, 这与HuggingGPT (433) 和Sasha (1069) 相当。这种固定令牌使用量的相似性并未导致显著的成本差异。对于可变部分, TaskSense包含了额外的组成部分, 例如语法

表6: 大语言模型（GPT-4）令牌计数细分。

阶段	模块	# 平均使用令牌数
Planning	1. Goal Description	558
	2. Examples	129/example
	3. Vocabulary Set	137/tool
	4. Grammar Rules	11/tool
Responding	1. Goal Description	187
	2. Execution Results	63/time interval

和示例，这些在HuggingGPT和Sasha中并不存在。具体来说，HuggingGPT缺少语法部分，而Sasha则缺少示例和语法两部分。平均而言，每条额外的语法规则会增加11个令牌的使用量，每个示例会增加129个令牌。尽管存在这些增加，我们方法的总体成本并未显著上升，且保持在可接受范围内，这使得我们的系统在实际应用中具有成本效益。

8 讨论

对基础模型工具的扩展性。当前的工具集仅包含特定任务的人工智能模型。许多多模态基础模型，例如LLaVA [34] 和 ImageBind [19], 能够用一个模型执行多样的下游任务。在未来的工作中，可以增强TaskSense以支持更复杂的工具依赖关系，并将其扩展以将这些多模态模型集成到工具集中，使其能够处理更广泛的任务。

动态计划适应策略。TaskSense提供了强大的兼容性，其动态计划适应策略可以定制。目前，TaskSense主要关注现实场景中传感器数据和执行结果的质量，采用结合执行前过滤和执行后选择的策略。未来的工作可以通过实施其他策略来扩展TaskSense以满足额外需求，例如基于执行延迟的路径选择以实现实时性能，或利用所有可用数据流进行多模态融合以提升下游任务性能 [39]。

降低大语言模型API调用成本。目前，TaskSense需要为每个新查询调用大语言模型API，导致不可避免的财务支出和时间开销。未来的工作可以引入缓存机制，以避免对类似查询的频繁API调用，并对计划执行结果采用更紧凑的表示形式。

9 结论

本文研究了利用大语言模型来协调传感器系统以处理复杂的用户查询。为实现此目标，本文提出了一种基于传感器语言定义的类翻译方法，以将人类意图解释为可执行计划。为提高执行鲁棒性，TaskSense采用了动态计划适配机制，根据环境因素的反馈来调整计划。我们在一个端到端应用设置中对TaskSense进行了原型实现，并使用四个数据集进行了测试。评估结果表明，与基线相比，TaskSense提供了高达3×的规划准确率和1.75× 的响应准确率。

致谢

本研究部分得到国家自然科学基金委员会（NSFC）项目62202407以及

香港研究资助局（RGC）优配研究金14214022、优配研究金14212323和主题研究计划T43-513/23-N的资助。

参考文献

[1] 2023. Mistral Inference. <https://github.com/mistralai/mistral-inference>,.

[2] 2024. 自主与传感器技术. <https://www.statista.com/outlook/tmo/artificial-intelligence/autonomous-sensor-technology/worldwide>,.

[3] 2024. Hello GPT-4o. <https://openai.com/index/hello-gpt-4o/>,.

[4] 2024. 国际劳工组织. <https://www.ilo.org/>,.

[5] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Floren- cia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. arXiv preprint arXiv:2303.08774 (2023).

[6] Alfred V Aho, John E Hopcroft, and Jeffrey D Ullman. 1973. On finding lowest common ancestors in trees. In Proceedings of the fifth annual ACM symposium on Theory of computing. 253–265.

[7] Cohere AI. 2023. Introduction to Text Embeddings. https://github.com/cohere-ai/notebooks/blob/main/notebooks/llmu/Introduction_Text_Embeddings.ipynb?ref=cohere-ai.ghost.io.

[8] 亚马逊网络服务. 2024. Amazon Bedrock: Build Generative AI Applications with Foundation Models. <https://aws.amazon.com/bedrock/>. Accessed: 2024-06-30.

[9] Home Assistance. 2024. Home Assistance: Awaken Your Home. <https://www.home-assistant.io/>.

[10] Daniel J Buysse, Martica L Hall, Patrick J Strollo, Thomas W Kamarck, Jane Owens, Laisze Lee, Steven E Reis, and Karen A Matthews. 2008. Relationships between the Pittsburgh Sleep Quality Index (PSQI), Epworth Sleepiness Scale (ESS), and clinical/polysomnographic measures in a community sample. Journal of clinical sleep medicine 4, 6 (2008), 563–571.

[11] 陈慧敏, Chaojie Gu, Lilin Xu, Rui Tan, Shibo He, and Jiming Chen. 2025. Listen to your face: a face authentication scheme based on acoustic signals. ACM传感器网络汇刊 21, 1 (2025), 1–23.

[12] 陈浩, 张一鸣, 张琪, 杨瀚涛, 胡晓萌, 马雪涛, 杨一凡, and 赵军波. 2023. Maybe only 0.5% data is needed: A preliminary exploration of low training data instruction tuning. arXiv preprint arXiv:2305.09246 (2023).

[13] C Craig, A Marshall, M Sjostrom, A Bauman, P Lee, D Macfarlane, T Lam, and S Stewart. 2017. International physical activity questionnaire-short form. 美国大学健康杂志 65, 7 (2017), 492–501.

[14] 崔宏伟, 杜宇阳, 杨群, 邵玉林, and 刘松昌. 2023. Llmind: Orchestrating ai and iot with llms for complex task execution. arXiv preprint arXiv:2312.09007 (2023).

[15] Ryandito Diandaru, Lucky Susanto, 唐子路, Ayu Purwarianti, and Derry Wijaya. 2024. What Linguistic Features and Languages are Important in LLM Translation? arXiv preprint arXiv:2402.13917 (2024).

[16] Hugging Face. 2024. FFmpeg: A complete, cross-platform solution to record, convert and stream audio and video. <https://www.ffmpeg.org/>.

[17] Hugging Face. 2024. Hugging Face Community. <https://huggingface.co/>.

[18] Jacopo Galli, D Meucci, Giampiero Salonna, R Anzivilino, Valentina Giorgio, M Trozzi, Stefano Settini, ML Tropiano, Gaetano Paludetti, and S Bottero. 2020. Use Of NBI for the assessment of clinical signs of rhino-pharyngo-laryngeal reflux in pediatric age: Preliminary results. 国际小儿耳鼻喉科杂志 128 (2020), 109733.

[19] Rohit Girdhar, Alaaeldin El-Nouby, 刘壮, Mannat Singh, Kalyan Vasudev Alwala, Armand Joulin, and Ishan Misra. 2023. ImageBind: One Embedding Space To Bind Them All. In 计算机视觉与模式识别会议.

[20] Christopher G Goetz, Barbara C Tilley, Stephanie R Shaftman, Glenn T Stebbins, Stanley Fahn, Pablo Martinez-Martin, Werner Poewe, Cristina Sampaio, Matthew B Stern, Richard Dodel, et al. 2008. Movement Disorder Society-sponsored revision of the Unified Parkinson’s Disease Rating Scale (MDS-UPDRS): scale presentation and clinimetric testing results. Movement disorders: official journal of the Movement Disorder Society 23, 15 (2008), 2129–2170.

[21] 韩佳明, 龚凯雄, 张艺源, 王佳琪, 张开鹏, 林达华, 乔宇, 高鹏, and 岳翔宇. 2024. Onellm: One framework to align all modalities with language. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 26584–26595.

[22] Jussi S Jauhiainen and Agustín Garagorry Guerra. 2024. Evaluating Students’ Open-ended Written Responses with LLMs: Using the RAG Framework for GPT- 3.5, GPT-4, Claude-3, and Mistral-Large. arXiv preprint arXiv:2405.05444 (2024).

[23] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C Park. 2024. Database-Augmented Query Representation for Information Retrieval. arXiv preprint arXiv:2406.16013 (2024).

[24] Melvin Johnson, Mike Schuster, Quoc V Le, Maxim Krikun, 吴永辉, 陈智峰, Nikhil Thorat, Fernanda Viégas, Martin Wattenberg, Greg Corrado, et al. 2017. Google’s multilingual neural machine translation system: Enabling zero-shot translation. Transactions of the Association for Computational Linguistics 5

- (2017), 339–351.[25] Evan King, Haoxiang Yu, Sangsu Lee, and Christine Julien. 2024. Sasha: creative goal-oriented reasoning in smart homes with large language models. ACM交互式、移动、可穿戴与普适技术汇刊 8, 1 (2024), 1–38.[26] Philipp Koehn. 2009. Statistical machine translation. 剑桥大学出版社.[27] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. 神经信息处理系统进展 35 (2022), 22199–22213.[28] Roman Koshkin, Katsuhito Sudoh, and Satoshi Nakamura. 2024. Transllama: Llm-based simultaneous translation system. arXiv preprint arXiv:2402.04636 (2024).[29] Labelbox. 2024. GPT-3.5: Models - OpenAI. <https://labelbox.com/product/model/foundry-models/gpt-3-5/>. Accessed: 2024-06-30.[30] Guillaume Lample and Alexis Conneau. 2019. Cross-lingual language model pretraining. arXiv preprint arXiv:1901.07291 (2019).[31] McMahon E Lawton. 2008. Brody instrumental activities of daily living scale (IADL). MaineHealth 108, 4 (2008), 2.[32] Xian Li, Ping Yu, Chunting Zhou, Timo Schick, Luke Zettlemoyer, Omer Levy, Jason Weston, and Mike Lewis. 2023. Self-alignment with instruction backtrans-lation. arXiv preprint arXiv:2308.06259 (2023).[33] Yaobo Liang, Chenfei Wu, Ting Song, Wenshan Wu, Yan Xia, Yu Liu, Yang Ou, Shuai Lu, Lei Ji, Shaoguang Mao, et al. 2023. Taskmatrix. ai: Completing tasks by connecting foundation models with millions of apis. arXiv preprint arXiv:2303.16434 (2023).[34] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual Instruc- tion Tuning. In NeurIPS.[35] Zhaoyang Liu, Zeqiang Lai, Zhangwei Gao, Erfei Cui, Zhiheng Li, Xizhou Zhu, Lewei Lu, Qifeng Chen, Yu Qiao, Jifeng Dai, et al. 2023. Controllm: Augment lan- guage models with tools by searching on graphs. arXiv preprint arXiv:2310.17796 (2023).[36] Ning Miao, Yee Whye Teh, and Tom Rainforth. 2023. Selfcheck: Using llms to zero-shot check their own step-by-step reasoning. arXiv preprint arXiv:2308.00436 (2023).[37] 微软. 2024. Azure OpenAI Service. <https://azure.microsoft.com/en-us/products/ai-services/openai-service>. [38] 英伟达公司. 2023. Jetson Orin Modules and Developer Kits. <https://www.nvidia.com/m/en-us/autonomous-machines/embedded-systems/jetson-orin/>[39] Xiaomin Ouyang, Xian Shuai, Yang Li, Li Pan, Xifan Zhang, Heming Fu, Sitong Cheng, Xinyan Wang, Shihua Cao, Jiang Xin, et al. 2024. ADMarker: A Multi-Modal Federated Learning System for Monitoring Digital Biomarkers of Alzheimer’ s Disease. In 第30届移动计算与网络国际年会议录. 404–419.[40] Xiaomin Ouyang, Xian Shuai, Yang Li, Li Pan, Xifan Zhang, Heming Fu, Xinyan Wang, Shihua Cao, Jiang Xin, Hazel Mok, et al. 2023. ADMarker: A Multi-Modal Federated Learning System for Monitoring Digital Biomarkers of Alzheimer’ s Disease. arXiv preprint arXiv:2310.15301 (2023).[41] Xiaomin Ouyang and Mani Srivastava. 2024. LLMsense: Harnessing LLMs for High-level Reasoning Over Spatiotemporal Sensor Traces. arXiv preprint arXiv:2403.19857 (2024).[42] Haojie Pan, Zepeng Zhai, Hao Yuan, Yaojia Lv, Ruiji Fu, Ming Liu, Zhongyuan Wang, and Bing Qin. 2023. Kwaigents: Generalized information-seeking agent system with large language models. arXiv preprint arXiv:2312.04889 (2023).[43] Poe. 2024. Poe平台. <https://poe.com/>[44] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis. arXiv preprint arXiv:2307.16789 (2023).[45] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2024. Toolformer: Language models can teach themselves to use tools. Advances in Neural Information Processing Systems 36 (2024).[46] Dhruv Shah, Błażej Osiński, Sergey Levine, et al. 2023. Lm-nav: Robotic navigation with large pre-trained models of language, vision, and action. In 机器人学习会议. PMLR, 492–504.[47] Azmeh Shahid, Kate Wilkinson, Shai Marcu, and Colin M Shapiro. 2011. Fatigue assessment scale (FAS). In 《STOP, THAT及其他一百种睡眠量表》. 施普林格, 161–162.[48] Yifei Shen, Jiawei Shao, Xinjie Zhang, Zehong Lin, Hao Pan, Dongsheng Li, Jun Zhang, and Khaled B Letaief. 2024. Large language models empowered autonomous edge ai for connected intelligence. 《IEEE通信杂志》 (2024).[49] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2024. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. Advances in Neural Information Processing Systems 36 (2024).[50] Shuyao Shi, Neiwen Ling, Zhehao Jiang, Xuan Huang, Yuze He, Xiaoguang Zhao, Bufang Yang, Chen Bian, Jingfei Xia, Zhenyu Yan, et al. 2024. Soar: Design and Deployment of A Smart Roadside Infrastructure System for Autonomous Driving. 载于第30届移动计算与网络国际年会议录. 139–154.[51] Amit Kumar Sikder, Leonardo Babun, Z Berkay Celik, Hidayet Aksu, Patrick McDaniel, Engin Kirda, 以及 A Selcuk Ulugac. 2022. 谁在控制我的设备? 面向共享智能家居环境的多用户多设备感知访问控制系统. ACM物联网汇刊 3, 4 (2022), 1–39.[52] Spotify. 2024. 近似最近邻算法库. <https://github.com/spotify/annoy>. [53] Jennifer Yee-man Tang, Andy Hau-yan Ho, Hao Luo, Gloria Hoi-yan Wong, Bobo Hi-po Lau, Terry Yat-sang Lum, 以及 Karen Siu-lan Cheung. 2016. 验证粤语版Zarit负担访谈简版 (CZBI-Short) 用于痴呆症照护者. 衰老与心理健康 20, 9 (2016), 996–1001.[54] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, 等. 2023. Llama: 开放且高效的基础语言模型. arXiv预印本 arXiv:2302.13971 (2023).[55] TP-link. 2024. TP-link智能家居社区. <https://community.tp-link.com/en/smart-home/>. [56] Geoffrey Vaquette, Astrid Orcesi, Laurent Lucat, 以及 Catherine Achard. 2017. 日常生活活动数据集: 一个用于在线识别的高语义活动数据集. 载于2017年第12届IEEE自动人脸与手势识别国际会议 (FG 2017). IEEE, 497–504.[57] Vzense. 2024. Vzense科技. <https://www.vzense.com/>. [58] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, 以及 Anima Anandkumar. 2023. Voyager: 一个基于大语言模型的开放式具身智能体. arXiv预印本 arXiv:2305.16291 (2023).[59] Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, 以及 Daxin Jiang. 2023. Wizardlm: 赋能大语言模型遵循复杂指令. arXiv预印本 arXiv:2304.12244 (2023).[60] Lilin Xu, Chaojie Gu, Rui Tan, Shibo He, 以及 Jiming Chen. 2023. MESEN: 利用多模态数据设计少标签的单模态人类活动识别. 载于第21届ACM嵌入式网络传感器系统会议录. 1–14.[61] Lilin Xu, Keyi Wang, Chaojie Gu, Xiuzhen Guo, Shibo He, 以及 Jiming Chen. 2024. GesturePrint: 为基于毫米波的手势识别系统启用用户识别. 载于2024年第44届IEEE分布式计算系统国际会议 (ICDCS). IEEE, 1074–1085.[62] Bufang Yang, Yunqi Guo, Lilin Xu, Zhenyu Yan, Hongkai Chen, Guoliang Xing, 以及 Xiaofan Jiang. 2024. SocialMind: 基于大语言模型的主动增强现实社交辅助系统, 具备类人感知以实现现场实时交互. arXiv预印本 arXiv:2412.04036 (2024).[63] Bufang Yang, Lixing He, Neiwen Ling, Zhenyu Yan, Guoliang Xing, Xian Shuai, Xiaozhe Ren, 以及 蒋欣. 2023. Edgefm: 利用基础模型在边缘进行开放集学习. 载于第21届ACM嵌入式网络传感器系统会议录. 111–124.[64] Bufang Yang, Lixing He, Kaiwei Liu, 以及 Zhenyu Yan. 2024. VIAssist: 为视障用户适配多模态大语言模型. arXiv预印本 arXiv:2404.02508 (2024).[65] Bufang Yang, Siyang Jiang, Lilin Xu, Kaiwei Liu, Hai Li, Guoliang Xing, Hongkai Chen, Xiaofan Jiang, 以及 Zhenyu Yan. 2024. Drhouse: 一个通过利用传感器数据和专家知识成果赋能大语言推理系统. ACM交互式、移动、可穿戴与普适技术汇刊 8, 4 (2024), 1–29.[66] Bufang Yang, Le Liu, Wenxuan Wu, Mengliang Zhou, Hongxing Liu, 以及 Xinbao Ning. 2023. BrainZ-BP: 一种利用脑部生物阻抗和心电图的无创无袖带血压估计方法. IEEE仪器与测量汇刊 (2023).[67] Huanqi Yang, Sijie Ji, Rucheng Wu, 以及 Weitao Xu. 2024. 你被追踪了吗? 发现大语言模型零样本轨迹追踪的力量! arXiv预印本 arXiv:2403.06201 (2024).[68] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, 以及 Yuan Cao. 2022. React: 在大语言模型中协同推理与行动. arXiv预印本 arXiv:2210.03629 (2022).[69] Jinliang Yuan, Chen Yang, Dongqi Cai, Shihe Wang, Xin Yuan, Zeling Zhang, Xiang Li, Dingge Zhang, Hanzi Mei, Xianqing Jia, 等. 2024. 作为构件的移动基础模型. 载于第30届移动计算与网络国际年会议录. 279–295.[70] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, 以及 Yu-Xiong Wang. 2023. 语言智能体树搜索在大语言模型中统一了推理、行动与规划. arXiv预印本 arXiv:2310.04406 (2023).[71] Chunting Zhou, Pengfei Liu, Puxin Xu, Srinivasan Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, 等. 2024. Lima: 对齐中的少即是多. 神经信息处理系统进展 36 (2024).[72] Yutao Zhu, Huaying Yuan, Shutong Wang, Jiongnan Liu, Wenhan Liu, Chen- long Deng, Zhicheng Dou, 以及 Ji-Rong Wen. 2023. 用于信息检索的大语言模型: 一项综述. arXiv预印本 arXiv:2308.07107 (2023).